

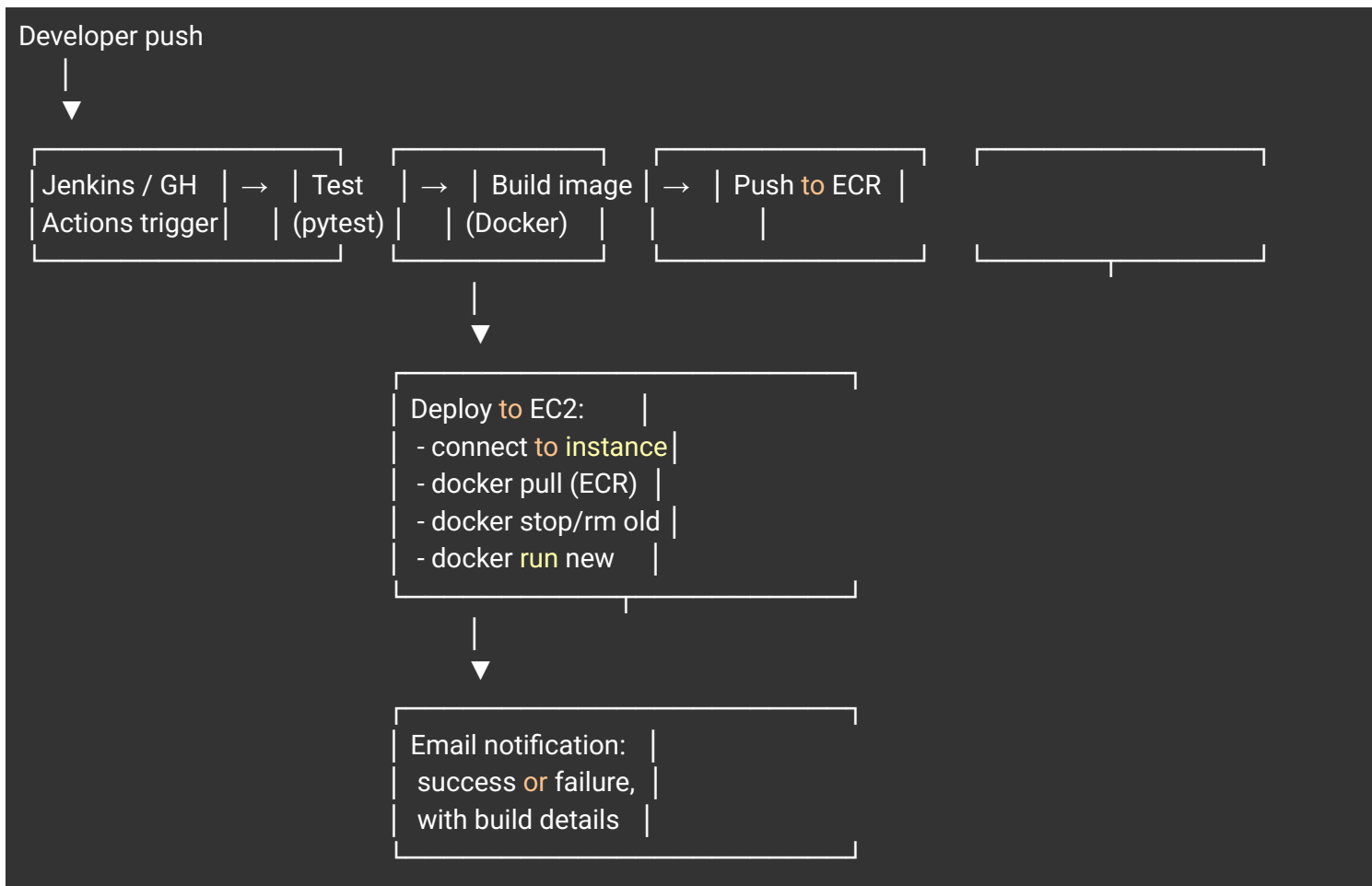
Graded Assignment on CI/CD Pipeline

Objective

Build a CI/CD pipeline using **either Jenkins or GitHub Actions** (your choice, not both) that automatically tests a Python Flask application, packages it into a Docker image, pushes that image to Amazon ECR, deploys it by running the container on an EC2 instance, and sends an **email notification with a customized message** reporting whether the run succeeded or failed.

This assignment tests whether you can wire together source control, automated testing, containerization, a container registry, a real compute target, and operational feedback (notifications) into one working pipeline the core CI/CD skill set expected in a DevOps role.

Architecture overview



Requirements

1. Application ([Github Repo LINK](#))

- Use a simple Python Flask web application with at least one health/status endpoint (e.g. `/health`) that the deployment step can use to confirm the container actually started correctly on EC2.
- Include a `requirements.txt` and a `pytest` test suite covering at least the core routes (success and failure cases).
- Include a `Dockerfile` that builds a runnable image of the application.

2. AWS prerequisites (set up manually before the pipeline runs)

- An **ECR repository** to hold the built images.
- An **EC2 instance** (Amazon Linux 2023 or Ubuntu) with:
 - Docker installed and running
 - An **IAM instance role** attached with permission to pull from ECR (`AmazonEC2ContainerRegistryReadOnly` or a scoped equivalent)
 - A security group allowing inbound traffic on the application's port (and SSH/port 22 only if you're using the SSH-based deploy method below)
- A way for the pipeline to reach the EC2 instance to trigger the deployment – choose **one**:
 - **SSH-based**: pipeline SSHes into the instance using a stored private key and runs the docker pull/run commands directly, or

3. Pipeline stages

Your pipeline (Jenkins **or** GitHub Actions) must implement, in order:

1. **Checkout** – pull the latest source code
2. **Install dependencies** – `pip install` from `requirements.txt`
3. **Test** – run the `pytest` suite; the pipeline must stop here (not proceed to build/deploy) if any test fails
4. **Build** – build the Docker image, tagged with the Git commit SHA (not just `latest`, so every deployed image is traceable to a commit)
5. **Push to ECR** – authenticate to ECR and push the tagged image
6. **Deploy to EC2** – connect to the EC2 instance (SSH or SSM) and:
 - pull the new image from ECR
 - stop and remove the currently running container (if any)
 - run the new container, mapping the application port
 - verify the app actually came up (e.g. curl the `/health` endpoint from the pipeline or from the instance itself) – this is your deploy-verification gate; a container that starts but crashes immediately should still be reported as a failed deployment

7. **Notify** – send an email reporting the outcome (see Section 5)

4. Triggers

- The pipeline must trigger automatically on every push to the **main** branch of the repository (GitHub webhook for Jenkins; **on: push** for GitHub Actions).

5. Email notifications with customized messages

This is graded separately from "a notification exists" – the email **content must differ meaningfully by outcome and include real build details**, not a generic template. At minimum:

On success, the email must include:

- A clear success indicator (e.g. subject line prefixed)
- The Git commit SHA and branch that was deployed
- The Docker image tag that was pushed to ECR
- The EC2 instance/target that was updated
- A link back to the pipeline run

On failure, the email must include:

- A clear failure indicator (e.g. subject line prefixed)
- **Which stage failed** (test / build / push / deploy) – not just "the pipeline failed"
- The Git commit SHA and branch
- A link to the pipeline run/logs so the failure can be investigated immediately without digging through the CI tool's UI first

Use environment/credential-stored SMTP settings (or your CI tool's native email step) – never hardcode email credentials in the pipeline file.

6. Secrets management

Store all sensitive values (AWS credentials or role ARN, SSH private key or EC2 host details) in your CI tool's credential store (Jenkins Credentials, or GitHub Secrets) – never commit them to the repository.

7. Documentation

Update the repository's **README.md** to cover:

- Prerequisites (AWS resources, IAM permissions, EC2 setup)
- How to configure the pipeline's required secrets
- How the deploy step connects to EC2 (SSH or SSM) and why you chose that method

- How to reproduce a deployment manually if the pipeline were unavailable

Deliverables

1. GitHub repository containing:
 - The Flask application, **Dockerfile**, and **pytest** test suite
 - The pipeline definition (**Jenkinsfile** or **.github/workflows/*.yaml**)
 - Updated **README.md**
2. Screenshots or a short screen recording showing:
 - A full pipeline run, all stages green, ending in a successful EC2 deployment
 - The success email received
 - At least one **intentionally broken run** (e.g. a failing test) showing the pipeline stopping early and the failure email received, with the correct failed-stage information in it
3. A text/Word/PDF file containing the repository link, submitted via Vlearn.

Grading rubric

Criterion	Weight
Pipeline stages present and passing in correct order (test gates build/deploy)	25%
Docker image built correctly and tagged with commit SHA	15%
Image successfully pushed to ECR	10%
Deployment to EC2 actually replaces the running container (not just a fresh install every time)	20%
Deploy-verification step (health check) genuinely gates success/failure	10%
Email notification – customized content, correct on both success and failure paths	15%
README documentation quality	5%

Submission instructions

- Ensure your assignment is fully completed and pushed to GitHub.
- Share the repository link in a text, Word, or PDF file.
- Submit the file via Vlearn.